

目标 backend, 不 莫型专 kernel 集合

硬 儻 收 斂 target、layout、
instruction atom runtime ownership。

PLATFORM

目标 target

支持 TileLang PrimFunc lowering、C codegen、
构 、部 、执行 错误 传 。

HARDWARE

分层 件抽象

HVX vector、HMX matrix、VTCM、DMA
worker 自 晰 界。

INTEGRATION

两 挑行 莫式

Standalone FastRPC用于发 ； embedded
resource lease用于现 runtime。

标 准

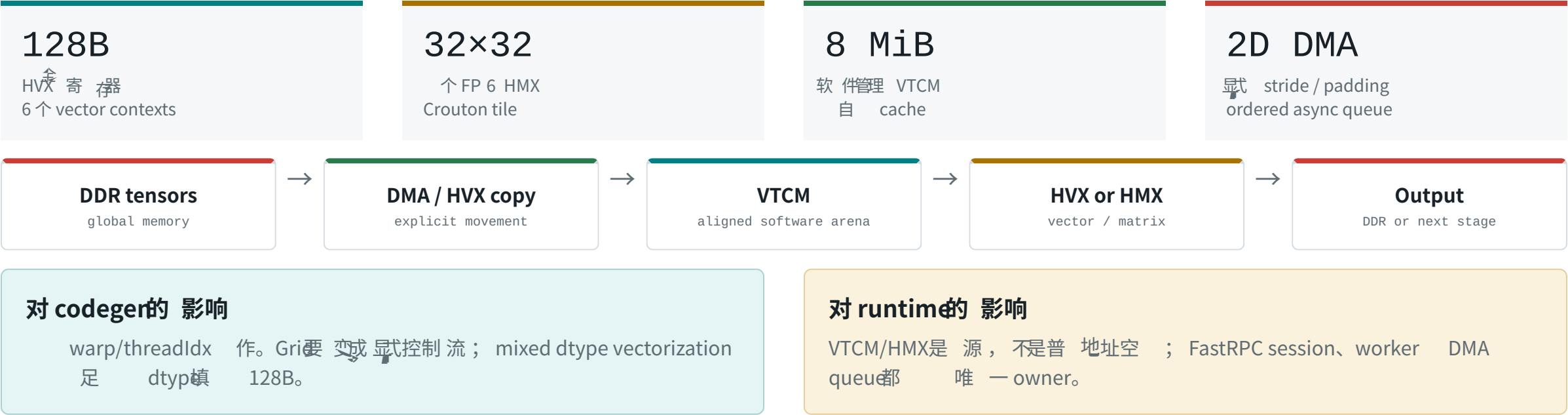
- **读**：generated C 中 看 VTCM 硬 件 用
- **可跑**：真实 果与 torch golden 一
- **可扩展**：新 op 不 需 复 制 完 runtime
- **可嵌入**：不 配 主 stock kernel

实 顺 序

- **Primitive** dtype、result repr 副 作用
- **Handwritten** 硬 件 的 性 上
- **Explicit TileLang** 复 现 loop、layout 赖
- **Generalize** 后 接 流 、worker autotune

Workload profiling 与 primitive 点 行 行：者 定 优 什么 后 者 定 TileLang 靠 表 什么。

Hexagon 不是 SIMD GPU



语义、度、指令与 源点 所有权 分开

L0	Operator semantics	shape、logical dtype、accum、tail epilogue	TileLang / workload
L1	Schedule + Layout	tile traversal、worker partition、`T.Layout`、buffer lifetime	TileLang
L2	Typed primitives	HVX value/repr、HMX clear/mma/convert/store dependency	Hexagon target
L3	Data movement	sync copy、async DMA、commit/wait VTCM plan	lowering + runtime
L4	Resource lifecycle	FastRPC session、VTCM/HMX acquire、DMA queue、worker pool	actual owner

关键判据 Standalone 时 TileLang runtime 是 owner, 嵌入 llama.cpp 时 主要是 owner, TileLang 只接收 源租 。

从 用 DSL 到 实现 cDSP 的五层 构

Hexagon是 CPU-like C backend的 sibling 不是 CUDA codeger的 变体。

L1	Frontend / TIR	`T.Kernel`、`T.copy`、`T.gemm`、`T.Layout` explicit atoms	tilelang/language
L2	Hexagon pipeline	CPU passes + Hexagon copy/fill/gemm/reduce + memory/layout transforms	src/hexagon/op
L3	C codegen	serialized grid、VTCM offsets、worker callback、device ABI	codegen_hexagon
L4	Hardware recipes	typed HVX/HMX、layout、VTCM、worker DMA recipes	tl_templates
L5	Runtime / embedding	FastRPC project + adb agent或 host-owned `tl_op_ctx`	tilelang/hexagon

codegen

发射 普 C 少量 runtime calls, 不 Crouton 配方。

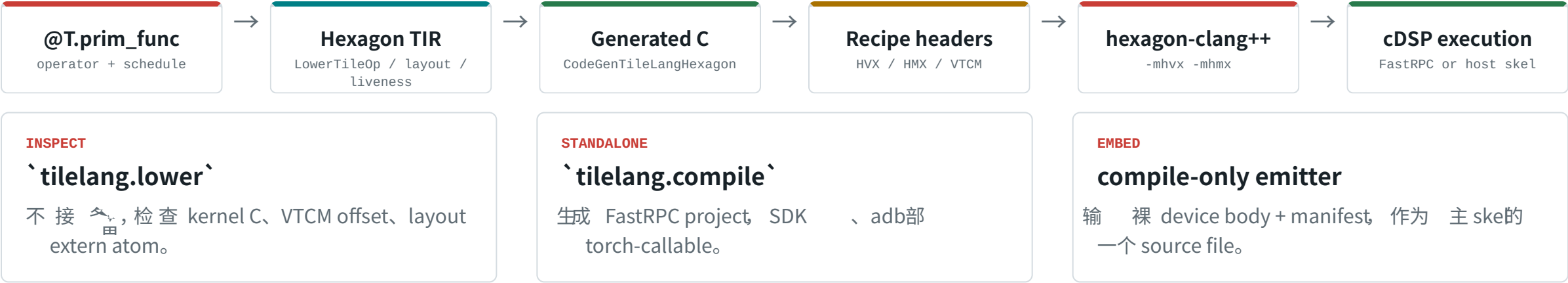
可 替换 recipe

硬件指令 header 中 代 保持 TIR接 稳定。

同一 generated body

既 被 standalone skel包 , 也 co-compile 主。

TileLang生成Hexagon算子的完整流程



参数 ABI 保持 host declaration order, 别称 shape 很容易掩盖错误的 alphabetical reorder。

`T.gemm` 定 primitives 之 的最 recipe

```
with T.Kernel(grid_n, grid_m, threads=1):
    A_sh = T.alloc_shared((BM, K), "float16")
    B_sh = T.alloc_shared((K, BN), "float16")
    C_sh = T.alloc_shared((BM, BN), "float16")
    T.copy(A_tile, A_sh)
    T.copy(B_tile, B_sh)
    T.gemm(A_sh, B_sh, C_sh, clear_accum=True)
    T.copy(C_sh, C_tile)
```

`T.alloc\_shared`
成 VTCM arena 的 对齐 静 offset, 发布 shared high-water。

`T.copy`
普 循 由 Hexagon vector codegen 变成 寄存器 HVX load/store。

`T.gemm`
Lower ``tl_hexagon_hmx_gemm``, template 完成 Crouton pack、MAC unpack。



上提、作 typed convert/store、Crouton layout、dependency 性基都 稳定, 且 explicit TileLang 路径仍 使用。

HMX intrinsic 与 `T.Layout` 暴露正 的低层 界



Activation layout

row-pair interleave, K对齐, `T.Layout` 编辑 `(m,k)` Crouton offset。

Weight layout

3 x3 tile, K-pair 与 channel 交；typed pack atom 直接地址。

Output layout

convert-state Crouton, 再 以 32-lane FP32 HVX 界输。

与 GPU intrinsic 的共同点

layout、load/mma/store atoms、显式 K loop、依赖 系都由 TileLang 看见，因 以融合。

必须保留的 件差异

HMX accumulator 核唯一且不 寻址；token 只表 存期，不伪成 register fragment。

同一个 generated body, 两 源 所 权

MODE A

Standalone FastRPC

- TileLang生成 IDL、ARM host、DSP skel、CMake
- TileLang申 VTCM/HMX/power, 启 persistent agent
- tensor marshal + adb + FastRPC, 直接 torch golden
- 合 backend 发、数 测 microbenchmark

MODE B

Embedded Host Skel

- generated body 作为源码 co-compile 主 skel
- 主 session、VTCM、DMA、worker HMX handle
- TileLang只 `tl\_op\_ctx`接受 同步 源租
- `run() == -` 即 stock kernel



源 由 申 , 就由 放。Embedded kernel 不 session、VTCM aren或 persistent worker pool。

-HMX /6 -HVX / VTCM / DMA的 度关

DMA	prefetch B0	prefetch B1	prefetch B2	prefetch B3	...	drain
HVX ×6	pack A	range B0	range B1	range B2	range B3	epilogue
HMX ×1	idle	compute 0	compute 1	compute 2	compute 3	release

num_workers=6

外层 blocks stride 配； 个 worker
VTCM slice, HVX 作 行。

单 HMX spinlock

pack/unpack 行，隐式 accumulator 议
串，不 伪 个 HMX。

Ordered D DMA

同 完成 stride padding queue、
descriptor cache policy留 runtime。

当前 缺口: `T.Pipelined`表 stage 依赖，Hexagon lowering 择 ordered DMA或 HVX fallback, queue/descriptor不 用户 DSL。

Q4/HMX: 用一个 Crouton 实现 layout、atom 与 schedule

目标：不使用 `T.gemm` 隐藏 K loop, 也不完 operator C helper。

5 B quant

4 × 128B HVX

64 B scales

32 × fp16

一个 576B native Q4_0 tile → 一个 32×32 FP 6 weight Crouton

Native atom 内部

load scale → low/high nibble → subtract8 → unpack → FP 6 multiply → Crouton stores。 个 quant vectors 同时保持 live, 提旨指令行。

```
# TileLang schedule stays visible
for tile in T.serial(tile_begin, tile_end):
    nt = tile // KT
    kt = tile % KT
    Q.dequant_tile(
        B_hmx, W,
        (nt, kt * 32, 0),
        (nt, kt, 0),
    )

for kt in T.serial(KT):
    E.mma_atom(acc, A_hmx, B_hmx, ...)
```

STAGE 1

Pack A once

FP3 → activation Crouton

STAGE 2

D DMA pad

576 B rows → aligned 640 B

STAGE 3

Parallel dequant

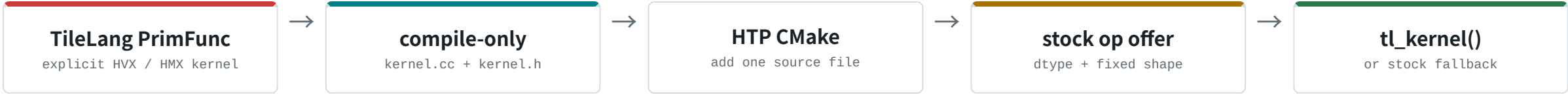
6 workers, disjoint tile ranges

STAGE 4

Explicit HMX

clear / K-mma / convert / store

最小：生成函数 直接 llama.cpp DSP skel



```
// generated header: plain C ABI
extern "C" int tl_kernel(
    void *vtcm, const void *src0,
    const void *src1, void *dst,
    int m, int n, int k);

// one offer before the existing DSP implementation
if (dtype == expected && m == M && n == N && k == K) {
    if (tl_kernel(vtcm, a, w, c, m, n, k) == 0)
        return 0;
}
return stock_kernel(...);
```

GENERATE

输出可直接链接的 C/C++

保留显式 layout intrinsic, kernel 不依赖 ggm 类，也不依赖 FastRPC session。

CO-COMPILE

只 source list

generated translation unit 使用 Hexagon clang 与 代码、同链接 DSP skel。

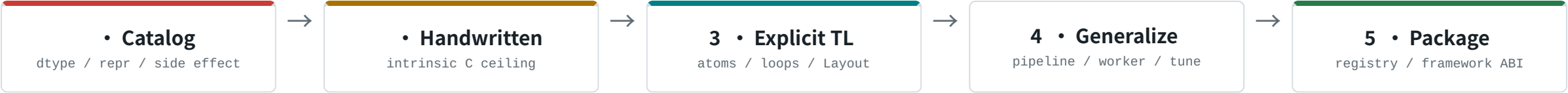
CALL + FALLBACK

先固定一个 用点文件

昔用 llama.cpp VTCM/HMX 源；未命中则失败时 续 stock 路径。

最小 收 模 forward 命中 generated symbol 输出 与 stock 一；他 shape 算子行为完全不变。

先对 件和类 型, 再自 动化算 子 度



TYPE FIRST

四 类型分开

logical dtype、compute dtype、hardware representation、storage dtype 不混用。

PROFILE IN PARALLEL

热、决定顺序

用占、shape覆盖、消除流量 复用价
定首地算子。

PARITY GATE

先 式再优化

TileLang explicit kernel 接 基，不上
提高层 o或 autotune。未

界： primitive 固 定 硬 件 义； recipe 合 primitives； schedule 定 loop、layout、pipeline worker。

FP32 add暴露了当前类型模型的缺口



HVX 期阻塞

- typed `HvxValue` 与 register count
- FP32 load/add/store generated path
- convert、predicate、tail、reduction

HMX 仍需

- typed convert mode, 删除 magic number
- 硬件 store 与 tensor dtype 层
- completion、operand lifetime、tail

Schedule primitives

- DMA D/ D + completion token
- worker id、barrier、join
- VTCM lifetime 与 double buffer

当前判： HMX core ； typed HVX、HMX output contract 统 一 conformance harness是下阶段底座。

重点、任务: 先 底座, 再做性能 和产品化

NOW · BLOCKER Typed primitives HVX FP3 load/add/store、 搬运 矩 ； HMX typed convert/store。	NEXT · BASELINE 手写性能 上 从热点 中 代表算子 ,统 一 ABI/ layout, 分离 compute、DMA 同 步成本 。	THEN · TILELANG Explicit parity 复现 atoms、loops、Layout, 再启用 pipeline、worker autotune。	PARALLEL · INTEGRATE 统一 桥梁 接口 与 generated kernel、用 registry、resource lease stock fallback。
WAVE 1 Primitive gate 类 正确、指令 中、device golden	WAVE 2 Kernel gate ceiling 与 瓶颈 因	WAVE 3 TileLang gate explicit kernel接 基	WAVE 4 Product gate 用 、package、CI、 本矩

期 关键路 径 typed HVX/HMX output → conformance tests → handwritten baseline → explicit TileLang parity。

从 backend 到 实际部署的路径

完 12 分钟；不用时仍离 fallback。

```
# 1. environment + offline compiler
./scripts/demo.sh check
./scripts/demo.sh lower
./scripts/demo.sh test-codegen

# 2. generic backend on device
./scripts/demo.sh matmul
./scripts/demo.sh workers

# 3. low-level cases
./scripts/demo.sh q4-standalone
./scripts/demo.sh q8-standalone

# 4. framework integration example
./scripts/demo.sh llama-walkthrough
./scripts/demo.sh inspect-llama
./scripts/demo.sh deploy-q4
./scripts/demo.sh bench-q4
./scripts/demo.sh chat-q4
./scripts/demo.sh restore-stock
```

5 分 精

`lower` → `matmul` → `deploy-q4` → `chat-q4` → `restore-stock`。只
硬件射、流程、Mode B 任务评估。

备 注

`lower` → `test-codegen` → `generate-q4`，然后展示 generated C、
manifest 档 raw logs。

收尾 查

`status` 确认 active skel 与 `.stock` SHA 56 相同。FastRPC 命令 从
作业 启。

从 例 向 primitive-first 工程 化

四、技

- 类型先行: logical、compute、hardware repr、storage 层
- 低层 atom 实用 layout dependency表 HMX/HV差
- 基 动 ceiling → explicit TileLang → 用 优 化
- 集 魔 一: standalone 与 embedded、用 package contract

集中交付目录

- `index.html`: 离 演 示 与 打
- `docs/`: 构、任 务 评 估、稿 runbook
- `scripts/`: 一条命令完成 演 示 阶 段
- `sources/`: TileLang 与 llama.cpp 源码 快照
- `artifacts/` / `results/`: skel、希 望 日 志

Primitive contract

typed and tested



Explicit parity

handwritten ceiling



General backend

pipeline / tune / package

真正的 交付是 复用的 typed primitives、compiler schedule、runtime contract framework package, 不是 定 算子集合 。